

# LICENCIATURA EM ENGENHARIA INFORMÁTICA

## INTELIGÊNCIA ARTIFICIAL

Análise de Imagens com CNNs

29447 - Ivo Ramoa (ivoramoa@ipvc.pt)

24611 - Diogo Fontes (diogofontes@ipvc.pt)

Docentes:

Jorge Ribeiro e Jackson Barreto

## ■ Índice

1. Introdução
2. Objetivos
3. Ferramentas Utilizadas
4. Análise do Dataset
5. Pré-Processamento dos dados
6. Arquitetura do Modelo
7. Compilação e Treino
8. Avaliação do Modelo
9. Conclusão
10. Referências

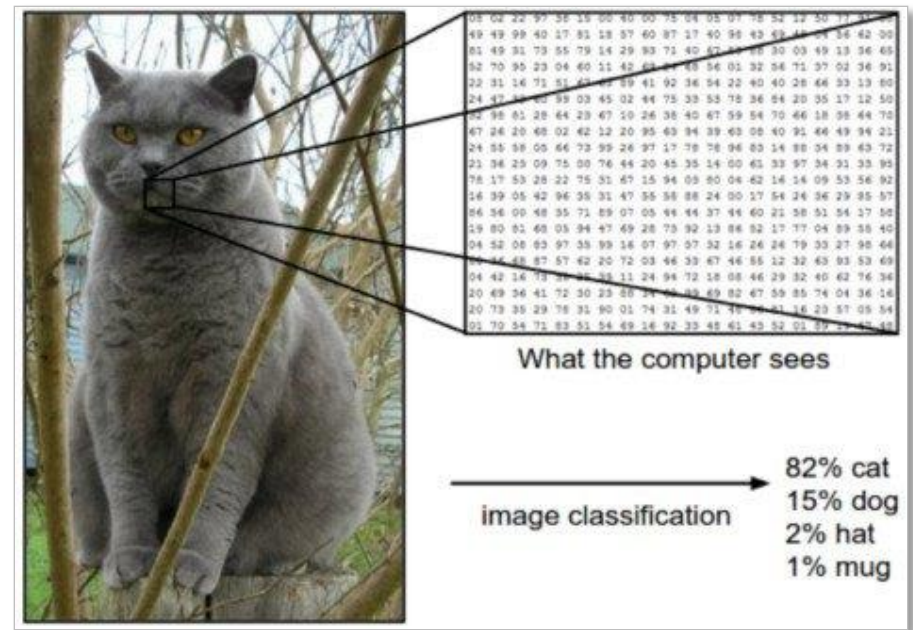
## ■ Introdução

Este trabalho tem como objetivo aplicar Redes Neurais Convolucionais (CNNs) à construção de um modelo preditivo baseado em dados de imagem, dando continuidade às abordagens supervisionadas já desenvolvidas nos TP2-C (Árvores de Decisão) e TP2-D (RNA com dados alfanuméricos).

As CNNs foram escolhidas pela sua elevada eficácia na análise de imagens, permitindo extrair padrões visuais relevantes para tarefas de classificação ou reconhecimento.

Utilizando um dataset de imagens, o modelo foi treinado e posteriormente testado com novos dados para simular a sua capacidade de generalização.

Este projeto reforça a aplicação prática de técnicas de deep learning no contexto da visão computacional, com recurso a ferramentas como Python e bibliotecas como Keras ou TensorFlow.



## ■ Objetivos

- Desenvolver um modelo preditivo utilizando Redes Neurais Convolucionais (CNNs).
- Aplicar técnicas de deep learning à análise e interpretação de dados de imagem.
- Utilizar um dataset de imagens para treinar e avaliar o modelo.
- Simular a aplicação prática do modelo com novos dados para prever classes ou características visuais.
- Consolidar conhecimentos sobre redes neurais profundas e visão computacional, recorrendo a ferramentas como Python, Keras e TensorFlow.

## ■ Ferramentas Utilizadas



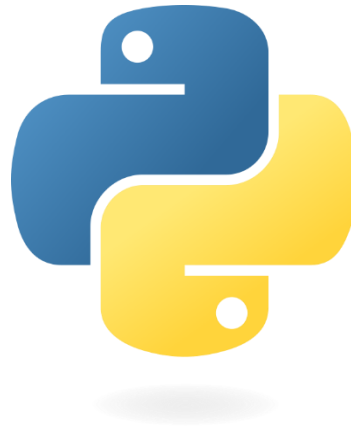
TensorFlow



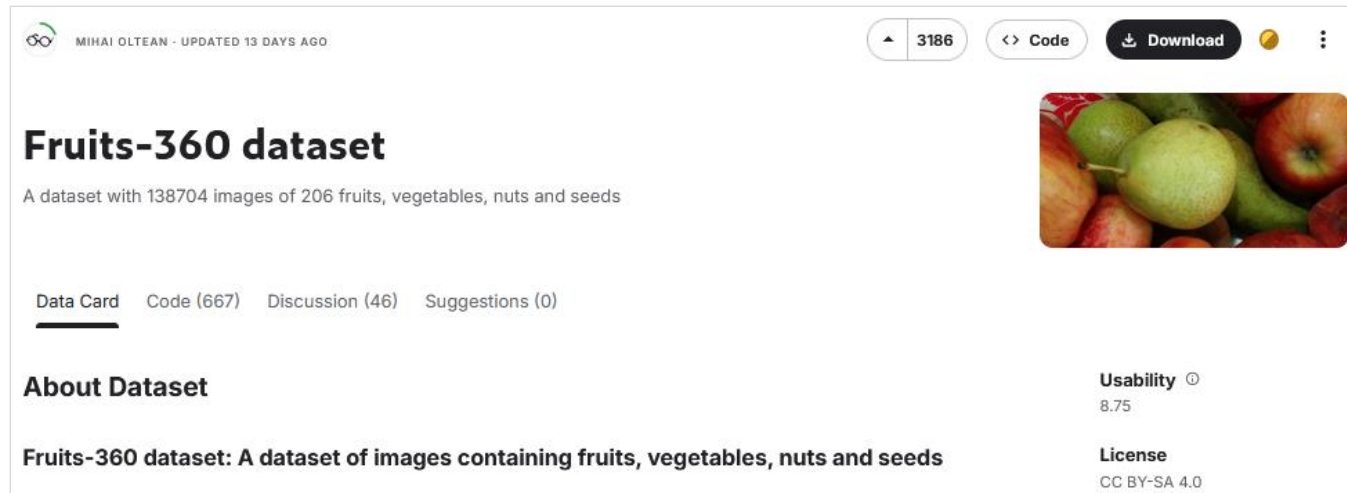
kaggle



ChatGPT



## ■ Análise do Dataset



O dataset utilizado neste trabalho foi o **"Fruits-360"**, composto por mais de **138.000 imagens de frutas, vegetais, sementes e frutos secos**, distribuídas por **206 classes**.

Este dataset é ideal para treinar modelos de **classificação de imagens** com **Redes Neurais Convolucionais (CNNs)**, dada a sua variedade e organização estruturada em conjuntos de treino e teste.

Link do dataset: <https://www.kaggle.com/datasets/moltean/fruits>

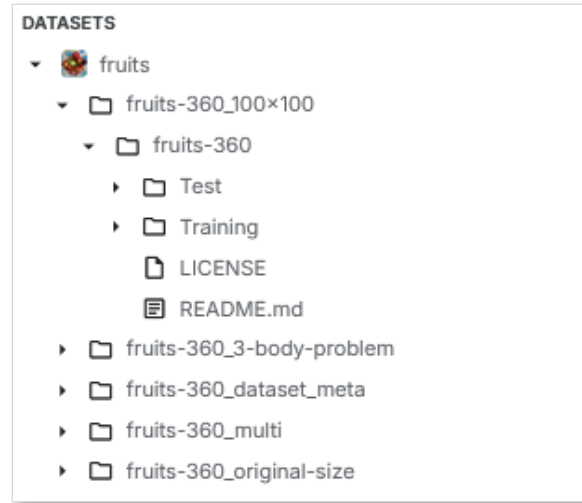
## ■ Análise do Dataset

O objetivo principal deste dataset é permitir a **classificação automática de alimentos** (frutas, vegetais, sementes e frutos secos) com base em imagens, utilizando redes neuronais convolucionais (CNNs). Este dataset é amplamente usado para **treino e avaliação de modelos de deep learning** na área da visão computacional.

Assim, é possível encontrar variáveis relevantes como:

- **Características Visuais:** Cor, forma, textura e orientação dos alimentos;
- **Categorias Alimentares:** Frutas, vegetais, frutos secos e sementes, agrupadas em 206 classes distintas;
- **Variedades:** Diferentes tipos dentro da mesma classe (ex: maçã Golden, maçã Pink Lady);
- **Formato da Imagem:** Tamanho padronizado (100×100 px) ou resolução original;
- **Condições de Captura:** Rotação do objeto, fundo branco, iluminação controlada.

## ■ Análise do Dataset



Neste trabalho foi usada a pasta **fruits-360\_100x100**, que já traz as imagens com **100x100 píxeis**, prontas para usar. Isso simplifica o pré-processamento e acelera o treino com redes convolucionais, mantendo todas as imagens no mesmo formato e reduzindo a complexidade.

As imagens estão separadas em **pastas de treino e teste**, o que facilita para ensinar o modelo e depois verificar o que ele aprendeu. Esta estrutura funciona bem com o ImageDataGenerator do Keras, que carrega as imagens e pode aplicar ajustes como normalização.

## ■ Pré-Processamento dos Dados

```
# Definição de Parâmetros
IMG_SIZE      = (100, 100)
BATCH_SIZE    = 32
NUM_EPOCHS    = 25
VAL_SPLIT     = 0.2
SEED          = 123
AUTOTUNE      = tf.data.AUTOTUNE

random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Começamos por definir alguns **parâmetros importantes** para preparar os dados antes do treino do modelo.

Estes incluem o **tamanho das imagens** (100 x 100 píxeis), o **tamanho do lote** (32 imagens por iteração), e o **número de épocas de treino** (25).

Também definimos uma **divisão dos dados** com 80% para treino e 20% para validação. Para garantir que os resultados sejam **reprodutíveis**, fixámos uma seed (SEED = 2025) que controla a aleatoriedade do processo.

Por fim, ativámos o **AUTOTUNE** do TensorFlow para otimizar automaticamente o carregamento dos dados durante o treino.

## ■ Pré-Processamento dos Dados

```
DATA_DIR = pathlib.Path('/kaggle/input/fruits/fruits-360_100x100/fruits-360')

assert DATA_DIR.exists(), f"{DATA_DIR} not found – check the dataset path!"

# Carregar dados de treino e validação
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATA_DIR / 'Training',
    labels='inferred',
    label_mode='categorical',|
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    validation_split=VAL_SPLIT,
    subset='training',
    seed=SEED
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATA_DIR / 'Training',
    labels='inferred',
    label_mode='categorical',
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    validation_split=VAL_SPLIT,
    subset='validation',
    seed=SEED
)
```

```
# Carregar classes
class_names = train_ds.class_names
NUM_CLASSES = len(class_names)
print("Classes:", NUM_CLASSES)

# Dataset de teste
test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATA_DIR / 'Test',
    labels='inferred',
    label_mode='categorical',
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False
)
```

Após definir os parâmetros iniciais, **carregámos as imagens do dataset** com a função **image\_dataset\_from\_directory** do TensorFlow, que associa automaticamente as imagens às suas classes.

Divimos os dados da pasta **Training** em dois subconjuntos: **80% para treino e 20% para validação**, garantindo consistência com uma **seed fixa**.

Por fim, carregámos as imagens de teste da pasta **Test**, mantendo a ordem original (shuffle = false) para facilitar a análise dos resultados.

## ■ Pré-Processamento dos Dados

```
def configure(ds):  
    return (ds.cache()  
            .shuffle(1024, seed=SEED)  
            .prefetch(AUTOTUNE))  
  
train_ds = configure(train_ds)  
val_ds   = configure(val_ds)
```

Para melhorar o desempenho durante o treino, **configurámos os dados de treino e validação com cache, baralhamento e pré-carregamento**. Isto permite um carregamento mais **rápido e eficiente**, aproveitando melhor os recursos do sistema enquanto o modelo está a treinar.

```
augmenter = tf.keras.Sequential([  
    tf.keras.layers.RandomFlip(mode='horizontal'),  
    tf.keras.layers.RandomRotation(0.10),  
    tf.keras.layers.RandomZoom(0.10),  
], name='augmentation')
```

Criámos uma camada de aumento de dados para ajustar o modelo a generalizar melhor. Foram aplicadas transformações simples como **espelhamento horizontal, rotações e zoom aleatório** nas imagens, simulando pequenas variações que podem acontecer no mundo real.

## Arquitetura do Modelo

```
base = tf.keras.applications.EfficientNetV2S(
    include_top=False,
    weights='imagenet',
    input_shape=IMG_SIZE + (3,),
    pooling='avg'
)
base.trainable = False

inputs = layers.Input(shape=IMG_SIZE + (3,))
x = augmenter(inputs)
x = tf.keras.applications.efficientnet_v2.preprocess_input(x)
x = base(x, training=False)
x = layers.Dropout(0.30)(x)
outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)

model = models.Model(inputs, outputs, name='EffNetV2S-Fruits360')
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

| Layer (type)                  | Output Shape        | Param #    |
|-------------------------------|---------------------|------------|
| input_layer_1 (InputLayer)    | (None, 100, 100, 3) | 0          |
| augmentation (Sequential)     | (None, 100, 100, 3) | 0          |
| efficientnetv2-s (Functional) | (None, 1280)        | 20,331,360 |
| dropout (Dropout)             | (None, 1280)        | 0          |
| dense (Dense)                 | (None, 206)         | 263,886    |

Total params: 20,595,246 (78.56 MB)  
Trainable params: 263,886 (1.01 MB)  
Non-trainable params: 20,331,360 (77.56 MB)

Para a arquitetura do modelo, utilizamos o **EfficientNetV2S**, um modelo pré-treinado em milhões de imagens (ImageNet), como base para extração de características visuais.

Congelámos os seus pesos para evitar que fossem alterados durante o treino.

Em seguida, adicionámos uma camada de **Dropout** para reduzir o risco de overfitting e uma **camada densa final com ativação softmax** para classificar as imagens pelas suas respetivas classes.

O modelo foi compilado com o otimizador **Adam**, a função de perda **categorical crossentropy** e a métrica de **accuracy**.

Link do Modelo: [https://storage.googleapis.com/tensorflow/keras-applications/efficientnet\\_v2/efficientnetv2-s\\_notop.h5](https://storage.googleapis.com/tensorflow/keras-applications/efficientnet_v2/efficientnetv2-s_notop.h5)

Fonte: Tan, M., & Le, Q. V. (2021). *EfficientNetV2: Smaller models and faster training* [Preprint]. **arXiv**. <https://arxiv.org/abs/2104.00298>

## Arquitetura do Modelo

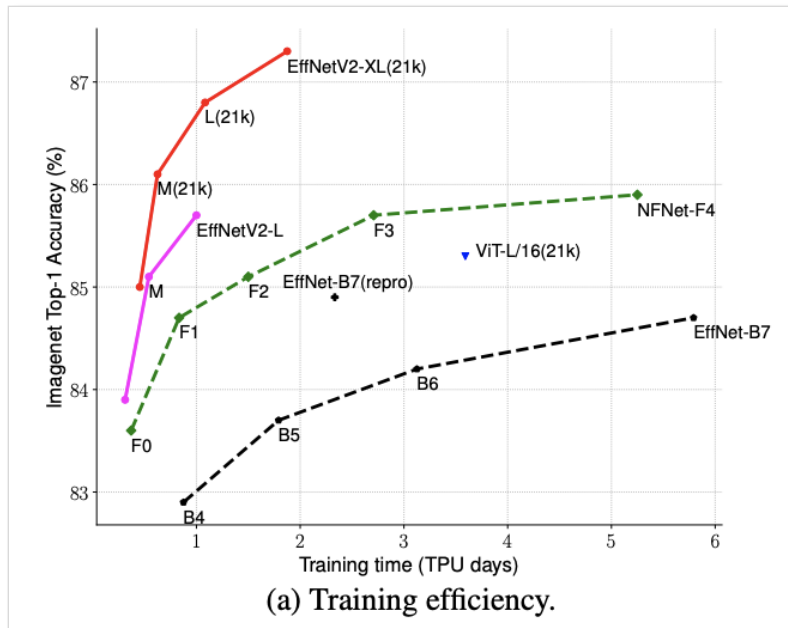


Imagem retirada:

<https://paperswithcode.com/method/efficientnetv2>

O **EfficientNetV2S** é uma arquitetura de rede convolucional moderna e otimizada, introduzida pela Google Research.

Faz parte da família **EfficientNetV2**, que combina **técnicas de compound scaling** com **training-aware neural architecture search (NAS)**, permitindo escalar profundidade, largura e resolução de forma coordenada.

A versão **"S" (Small)** é um dos modelos mais leves da série, projetado para obter **alta precisão com baixo custo computacional**.

Utiliza **camadas MBConv (Mobile Inverted Bottleneck)** e **Fused-MBConv**, que melhoram a eficiência computacional e a velocidade de treino. Pré-treinado no dataset **ImageNet**, o EfficientNetV2S é amplamente utilizado como **modelo base em Transfer Learning** para tarefas de classificação de imagens em grande escala.

Fonte: Tan, M., & Le, Q. V. (2021). *EfficientNetV2: Smaller models and faster training* [Preprint]. **arXiv**. <https://arxiv.org/abs/2104.00298>

## ■ Compilação e Treino

```
callbacks = [  
    tf.keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True),  
    tf.keras.callbacks.ReduceLROnPlateau(patience=3)  
]  
  
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=NUM_EPOCHS,  
    callbacks=callbacks)
```

Para tornar o treino mais eficiente, utilizámos duas **callbacks** que ajudam a melhorar o desempenho do modelo. A **EarlyStopping** permite interrompre o treino automaticamente se a validação deixar de melhorar durante algumas épocas, evitando o overfitting. Já a **ReduceLROnPlateau** ajusta a taxa de aprendizagem quando o progresso desacelera, permitindo que o modelo continue a melhorar de forma mais estável.

## ■ Compilação e Treino

```
base.trainable = True
for layer in base.layers[:-40]:
    layer.trainable = False

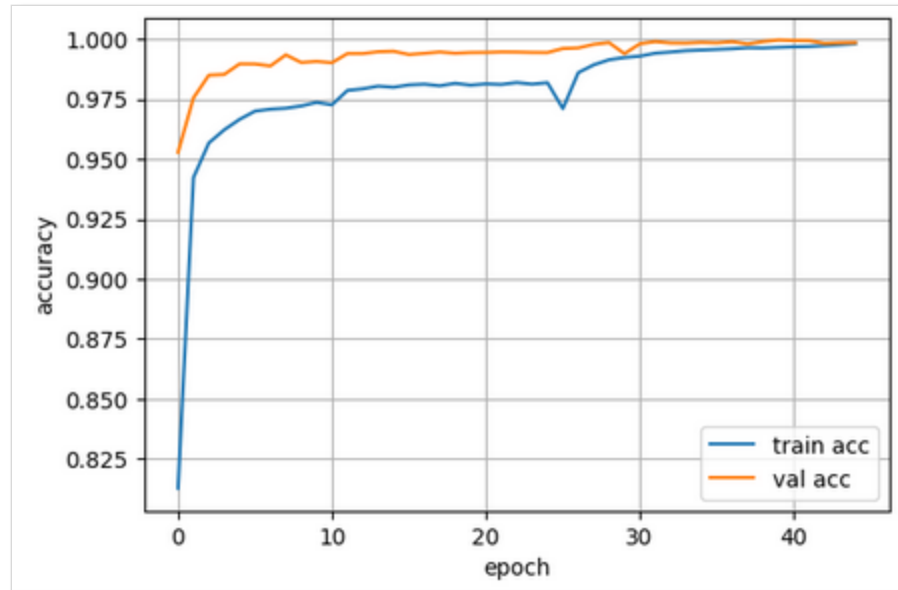
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy'])

fine_tune_epochs = 20
history_ft = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=fine_tune_epochs,
    callbacks=callbacks)
```

Após o treino inicial, ativámos o **fine-tuning** para melhorar ainda mais o desempenho do modelo. Para isso, desbloqueámos as últimas 40 camadas do EfficientNetV2S, permitindo que fossem ajustadas com os nossos dados.

Recompilámos o modelo com uma **taxa de aprendizagem menor**, para garantir que as atualizações fossem suaves, e voltámos a treinar por mais algumas épocas. Este processo ajuda o modelo a adaptar melhor os seus pesos às particularidades do nosso dataset, refletindo a sua capacidade de generalização.

## ■ Avaliação do Modelo



Os resultados obtidos demonstram um excelente desempenho do modelo. A precisão do conjunto de treino aumentou de forma consistente, **ultrapassando os 97%** após poucas épocas e aproximando-se do **100% no final**.

A precisão na validação manteve-se sempre elevada, atingindo valores **muito próximos de 99%**, o que indica que o modelo conseguiu generalizar bem e não sofreu de overfitting significativo.

## ■ Avaliação do Modelo

```
test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATA_DIR / 'Test',
    labels='inferred',
    label_mode='categorical',
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False)

test_ds = test_ds.cache().prefetch(AUTOTUNE)

loss, acc = model.evaluate(test_ds)
print(f"Test accuracy: {acc:.3%}")
```

```
Found 34711 files belonging to 206 classes.
1085/1085 ————— 49s 45ms/step - accuracy: 0.9895 - loss: 0.0399
Test accuracy: 98.897%
```

Por fim, avaliámos o desempenho final do modelo no conjunto de **teste**, composto por imagens nunca vistas durante o treino. Para garantir rapidez e eficiência, os dados foram cacheados e pré-carregados com **prefetch**.

Após a avaliação, o modelo atingiu uma **elevada precisão no teste**, confirmando que generaliza bem para novos dados e reforçando a qualidade da abordagem utilizada.

## ■ Avaliação do Modelo



Para verificar visualmente a performance do modelo, seleccionámos um conjunto de imagens de teste e comparamos as suas **classes reais** com as **previsões feitas pelo o modelo**.

As imagens foram apresentadas com o nome da classe verdadeira (T) e de prevista (P). Os títulos estão a **verde** quando a previsão foi correta e a **vermelho** quando houve erro.

Esta visualização permite confirmar que o modelo acerta na maioria dos casos e identificar facilmente as situações em que falha.

## ■ Conclusão

Neste trabalho, foi desenvolvido um modelo de classificação de imagens utilizando **Redes Neurais Convolucionais com Transfer Learning**, aplicado ao dataset **Fruits-360**.

Através da utilização do modelo pré-treinado **EfficientNetV2S**, combinado com técnicas de **data augmentation** e **fine-tuning**, foi possível alcançar uma precisão muito elevada, tanto nos dados de validação como no conjunto de teste.

Os resultados demonstraram uma excelente capacidade de generalização do modelo, com elevada fiabilidade na distinção entre diferentes tipos de frutas e vegetais.

A aplicação de estratégias como **EarlyStopping** e **ReduceLROnPlateau** contribuiu para um treino mais eficiente e controlado.

Este projeto reforça a eficácia das abordagens baseadas em **deep learning e transferência de conhecimento**, especialmente quando aplicadas a conjuntos de imagens bem estruturados como o Fruits-360.

## ■ Referências

- Mihai Oltean (2025): <https://www.kaggle.com/datasets/moltean/fruits>
- Modelo: [https://storage.googleapis.com/tensorflow/keras-applications/efficientnet\\_v2/efficientnetv2-s\\_notop.h5](https://storage.googleapis.com/tensorflow/keras-applications/efficientnet_v2/efficientnetv2-s_notop.h5)
- Imagem da Arquitetura: Papers with Code. (n.d.). *EfficientNetV2*. Papers with Code. Retrieved June 20, 2025, from <https://paperswithcode.com/method/efficientnetv2>
- EfficientNet V2: Tan, M., & Le, Q. V. (2021). *EfficientNetV2: Smaller models and faster training* [Preprint]. arXiv. <https://arxiv.org/abs/2104.00298>

o teu • de partida



[www.ipvc.pt](http://www.ipvc.pt)